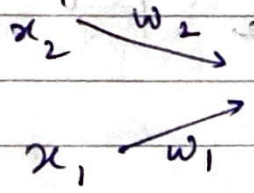
__/__/____

DL Gen AI - Week-2

L1 Universal Approximation Theorem

- LR recap



NN

Input → Hidden → Output

$a = g(\Sigma w_i x_i)$

"A LR is weighted sum of input features.

$\hat{y} = w_0 + w_1 x = \Sigma w_i x_i$

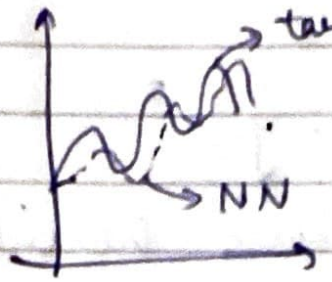
$a = \Sigma \rightarrow g$ (non linear activation func")

Σ → linear

- UAT → A feedforward network with a single hidden layer & a non-linear, bounded, continuous activation func" can approximate any continuous func" on a compact domain to arbitrary accuracy.

→ capacity → existence of params for better results

→ not a training guarantee, depth is not req., but it improves accuracy



each activated neurons acts like a soft step. Weighted sum forms the piece-wise smooth curves. ↑ hidden units to better approximate results.

- LR captures max hyperplanes. NN has flexible func^n families. UAT says about learnability, but not on generalizn^n.

L2 Forward Pass & Loss Functions

- Process of taking input x & feed it thru network layer by layer, each final predic^n $\hat{y}$.

$x = a^{[0]} \rightarrow z^1 = w^1 a^0 + b^1 \rightarrow g \ldots \rightarrow y = a^{[L]}$

every layer has -

1. linear combina^n — weighted sum of all neurons in the layer.
2. non-linear activa^n — apply activa^n func^n element-wise.

final $a^{[L]}$ is the predic^n $\hat{y}$.

- A loss func^n $L(y, \hat{y})$, returns error. We try to minimize this.

- for reg.

$$MSE = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

↳ penalizes large errors heavily.

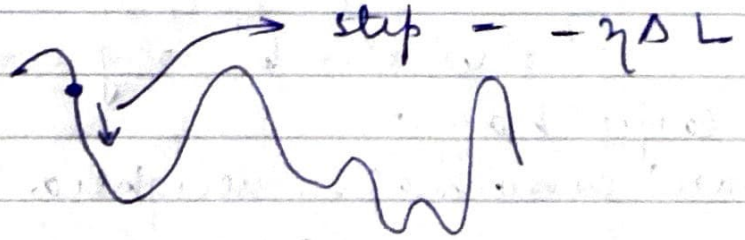
- for classificⁿ

Cross entropy, $L = -\sum_{c=1}^{C} y_c \log(\hat{y}_c)$

↳ penalizes the model if it is wrong & yet confident.

y_c: 1 or 0; $\hat{y}_c$: predicted probab.

L3 Gradient descent



direcⁿ of steepest ascent is gradient (∇L)
so we take steps in opp. direcⁿ

- if $\Delta L \rightarrow \eta$ (step) is too small, it will take forever, if it is too big, it overshoots the valley.

$$\theta_{new} = \theta_{old} - \eta \Delta L(\theta_{old})$$

(at each step, we move in the direc^n of steepest ascent)

- global minima → lowest pt., optimal solu^n
  local minima → valley-that traps the algo., prevents it to reach global minima.
-

$$\hat{y} = mx + c, \quad \theta = \{m, c\}$$

$$L(m,c) = \frac{1}{N}\sum(\hat{y}_i - y_i)^2$$

update

$$m = m - \eta \frac{\partial L}{\partial m}$$

$$c = c - \eta \frac{\partial L}{\partial c}$$

- refer to lectures slides → for diff.
  Batch GD, Stochastic GD, Mini-batch GD.
  entire dataset → single pt. → mini-batch of data.
  this is preferred in most DL applications
  efficient in terms of speed, convergence, & resources required for computa^n.

L4 Customer Churn Predic^n: Overview

- customer churn problem statement
- we analyze, engineer, handling → pre. build, evaluate, interpret, design

- business fact — retaining customer is much cost-effective than acquiring new ones.
- target is → binary → {0, 1} whether or not the customer will churn next month.
- we have several features for services, usage patterns, demographics, etc.
- metrics → precision, recall, F1, AUC-ROC.

L5 Churn: Data Processing

- kaggle churn dataset.
- imports, reproducibility, data loader, put it in df
- do data cleaning → outliers, missing values
- EDA → to understand & visualize patterns, check feature correlation.
- FE → normalization, create bins, remove bad fields, do class balancing.

L6 Neural Network Package Imports

- import torch.nn as nn
  - optim.lr_scheduler
  - metrics
  - optim

adam as optimizer

classification_report, roc_auc_score, confusion_matrix

ReduceLROnPlateau

- nn.functional as F

L7 Neural Network Implementation

- class ChurnPredictor
  - def __init__(input, hidden, dropout)
    - linear layer
      - batch normalization
        - activation (ReLu) ← max(0, a)
          - dropout
    - ← applying sigmoid non-linearity (output layer)
      - ↳ getting single logit to get probabilities

nn.Sequential ⇒ becomes an independent module

finally, wts. are initialized

FC → Fully connected

L8 Churn: Weight Initialization

def - initialize ⇒ make a custom one.

if isInstance (module, nn.Linear)

popular choice for FC layers

//use Xavier/Glorot initialization

set all bias to 0.

↳ this avoids any arbitary offset in the beginning

forward pass

↳ self.network ⇒ it passes all my layers into the network.

L9 Churn: Training & Validation Setup

- input size ⇐ no. of features

create the model ⇒ call class ChurnPredictor

move to GPU

model.

p.numel() for p in parameters()

↳ gives the total no. of parameters for this model.

from predictions ⇒ go and select the predicted class.

// if here your accuracy isn't good, first go and play with the learning rate.

L10 Chuan: Training Loop Implementation

number of epochs
keep the best validation loss
keep the training & validation losses
better to keep ash early stopping
criteria based on at certain threshold

train_loss ⇒ every thing (all steps)
similarly,
val_loss

scheduler.step() → to handle the learning rate based on the validation loss

keep all the metrics to later judge how our model has performed.

L11 Chunn: Model Training

- based on initializa^n, wait for the convergence.
- never try to visualize them —
  1. training / val loss per epochs
  2. accuracy over epoch
  3. learning rate " "
  4. overfitting check
- play with learning rate value.

L12 Backpropaga^n: Simple Scalar Chain

↳ it helps us to compute the gradients

- how will the small δ in param. affect the final loss.

$L(y(x))$, compute $\frac{dL}{dx}$

$x \xrightarrow{\frac{dy}{dx}} y(x) \xrightarrow{\frac{dL}{dy}} L(y)$

$$\frac{dL}{dx} = \frac{dL}{dy} \times \frac{dy}{dx}$$

Chain rule

influence of x on L is product of the local influences along the path

L is wrt y
y is wrt x
} we get $\frac{dL}{dx}$

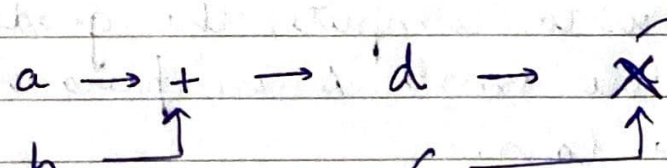
- for vector funcns, $y = f(x)$ & $L = g(y)$, the derivatives become Jacobian matrices & chain rule become matmul.

$$\boxed{\nabla_x L = (J_y L)(J_x y)}$$

- Jacobian matrix → contains all first-order partial derivatives of a vector valued func". It represents funcns bcz-linear approximan & sensitivity towards output.

- $L = (a+b) \cdot c$

$d = a+b$ & $L = d \cdot c$



$$L = (a+b) \cdot c$$

Forward pass → start with input values
backprop.

$L = (a+b) \cdot c$, $a = 2$, $b = 3$, $c = 6$

$\frac{dL}{dL} = 1$ $\quad\quad$ $\frac{dL}{dc} = \frac{dL}{dL} \times \frac{dL}{dc} = 1 \cdot d = 5$

$\frac{dL}{dd} = \frac{dL}{dL} \times \frac{dL}{dd} = 1 \cdot c = 6$

L13 **Back Prop : MLP** → multi-layered perceptron

- goal is to calculate the gradient of loss L wrt every wt. $W_{ij}^{[l]}$ & bias $b_i^{[l]}$

$$a^{[l-1]} \xrightarrow{W^{[l]}, b^{[l]}} z^{(l)} \xrightarrow{g(\cdot)} a^{[l]} \rightarrow L$$

for wts. $W^{[l]}$, $\frac{\partial L}{\partial W^{[l]}}$:

$$\boxed{\frac{\partial L}{\partial W^{[l]}} = \frac{\partial L}{\partial a^{[l]}} \times \frac{\partial a^{[l]}}{\partial z^{[l]}} \times \frac{\partial z^{(l)}}{\partial W^{[l]}}}$$

upstream · activation · local

- derivation of wt. update rule.

1. $\delta^l = \frac{\partial L}{\partial z^{(l)}}$

$$\delta^l = \frac{\partial L}{\partial a^{(l)}} \times \frac{\partial a^{(l)}}{\partial z^{(l)}}$$ → element-wise multiplication

$$\frac{\partial L}{\partial a^{(l)}} = \left(W^{[l+1]}\right)^T \delta^{[l+1]}$$

$$\Rightarrow \boxed{\delta^{[l]} = \left(\left(W^{[l+1]}\right)^T \delta^{[l+1]}\right) \odot g'(z^{[l]})}$$

for the output layer, it simplifies to -

$$\delta^{[l]} = \nabla_{a^l} L \odot g'(z^{[l]})$$

2. now we have to calculate the gradients for wts. & biases.

$$\boxed{\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)} \left(a^{(l-1)}\right)^T}$$

$$\boxed{\frac{\partial L}{\partial b^{(l)}} = \delta^{(l)}}$$

now, these gradients can be used to update our parameters.

L14 XOR Problem: Intro

- XOR ⇒ not linearly seperable
  ⇓
  there is no single straight line which will put two classes separate.
  ⇓ here perceptron fails.

$$f(x_1, x_2) = w_1 x_1 + w_2 x_2 + b$$

$$\hat{y} = \sigma\left(f(x_1, x_2)\right)$$

decision boundary ⇒ $f(x_1, x_2) = 0$
⇓ 0.5

eq^n, if separates 2 classes is –

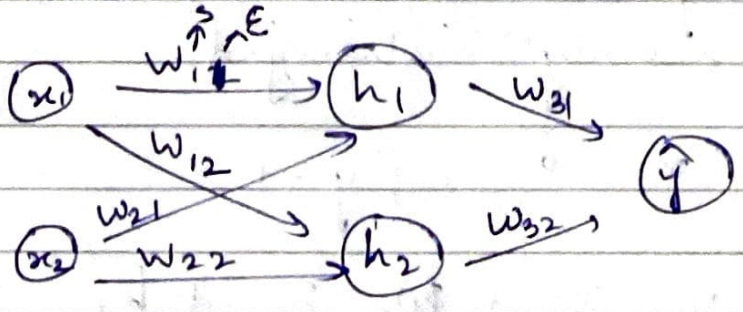
$$w_1 x_1 + w_2 x_2 + b = 0$$

for our model to solve XOR →

$(0,0) = b \approx 0$.
$(0,1) = w_2 + b \approx 1$
$(1,0) = w_1 + b \approx 1$!
$(1,1) = w_1 + w_2 + b \approx 0$.

but this can't be achieved ever.

- We have to use non-linear activa^n



forward pass

$$z_1 = w_{11} x_1 + w_{21} x_2 + b \Rightarrow h_1 = \sigma(z_1)$$
$$z_2 = w_{12} x_1 + w_{22} x_2 + b \Rightarrow h_2 = \sigma(z_2)$$
$$z_3 = w_{31} h_1 + w_{32} h_2 + b \Rightarrow \hat{y} = \sigma(z_3)$$

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

$$L(y, \hat{y}) = -[y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$$

↳ binary cross entropy loss

L15 XOR : Forward Pass

- Eg. $x = [1, 0]^T$; $y = 1$

$$W_1 = \begin{pmatrix} W_{11} & W_{12} \\ W_{21} & W_{22} \end{pmatrix} = \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}$$

$$b_1 = (b_1, b_2) = [0, 1]$$

$$W_2 = [W_{31}, W_{32}] = [2, -1]$$

$$b_2 = (b_3) = -1$$

Remember, when neural network is actually trained, that time, this will be randomly initialized.

$x_1 = 1$; $x_2 = 0$, $y = 1$

$z_1 = W_{11} x_1 + W_{21} x_2 + b_1 = 1$

$z_2 = 0$

$h_1 = \sigma(z_1) = 0.7$

$h_2 = \sigma(z_2) = 0.5$

$z_3 = -0.038$; $\hat{y} \approx +0.491$

$L \doteq 0.712$

$\frac{\partial L}{\partial W_{ij}}$ $\Longrightarrow$ we have to backpropagate now to get all upto W_{ij} using

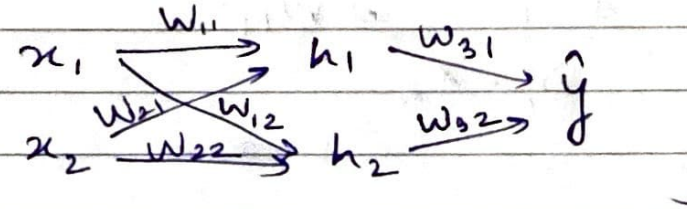
$$W_{new} = W_{old} - \alpha \frac{\partial L}{\partial W_{old}}$$

L16 XOR: Backpropagation

Recall $\delta^{(l)} = \frac{\partial L}{\partial a^{(l)}} \odot \frac{\partial a^{(l)}}{\partial z^{(l)}}$

$\leftarrow \left(W^{(l+1)}\right)^T \delta^{(l+1)}$; $\rightarrow g'(z^{(l)})$ activation derivative

↳ error propagated back from next layer.



} our XOR problem

recall the formulas.

- impt. derivative
  - $\rightarrow \sigma'(z) = \sigma(z)\left(1 - \sigma(z)\right)$
  - $\rightarrow \frac{\partial L}{\partial \hat{y}} = \frac{\hat{y} - y}{\hat{y}(1-\hat{y})}$
- chain rule stuff.
  - → calculate gradient backward.

$$\frac{\partial L}{\partial z_3} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_3} = \frac{\hat{y} - y}{\hat{y}(1-\hat{y})} \cdot \left(\hat{y}(1-\hat{y})\right)$$

$$\delta_3 = (\hat{y} - y)$$

$$\frac{\partial L}{\partial W_{31}} = \frac{\partial L}{\partial z_3} \cdot \frac{\partial z_3}{\partial W_{31}} = \delta_3 \cdot h_1$$

$$\frac{\partial L}{\partial W_{32}} = \delta_3 \cdot h_2$$

$$\frac{\delta L}{\delta z_1} = \frac{\delta L}{\delta z_3} \cdot \frac{\delta z_3}{\delta h_1} \cdot \frac{\delta h_1}{\delta z_1} = \delta_3 \cdot w_{31} \cdot \sigma'(z_1)$$

$$\frac{\delta L}{\delta z_2} = \delta_3 \cdot w_{32} \cdot \sigma'(z_2)$$

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_{11}} = \delta_1 \cdot x_1$$

$$\frac{\partial L}{\partial w_{22}} = \delta_1 \cdot x_2$$

now just plugin the mathematical values to get your gradients. Remember, you can calculate these things (layer-by-layer), from outermost to innermost.

Now, as per the update rule.

$$w_{new} = w_{old} - \alpha \frac{\partial L}{\partial w}$$

↙ learning rate

just get the new values.

- the hidden layer is able to transform the input data into a new representation which is linearly separable.

L17 Fashion MNIST Classifica^n: Overview

- build a MLP for MNIST (handwritten digits) inspired by
- image classifica^n problems (CV intro)
- 10 classes
- input (784 = 28×28) ← flattened
  - ↓
  - hidden 1 (128 + ReLu)
  - ↓
  - hidden 2 (64 + Relu)
  - ↓
  - output (10 classes + softmax).
- use dataloader to bring the dataset.
- specify the batch size to implement mini-batch.
- keep shuffle = True, so that the model does not learn the order of the data pts.
- now move to data visualiza^n & see how our dataset looks like.

L18 MNIST: MLP Implementa^n

- define new class FashionMLP

28×28 ⟹ flatten 1D vector

__/__/___

hidden layers helps you to learn the abstract features of the images.

782
↓
128
↓
64
↓
output → 10 class

funneling down

- between layers, add Relu activation. Then, apply dropout optionally.

L19 MNIST: Forward Pass

- input tensor (1, 28, 28)
output (batch-size, num-classes)

flatten (784) ← 28×28
↓
hidden 1 → 128 → relu → dropout
↓
hidden 2 → 64 → relu → dropout
↓
output → 10 ⟹ (32, 10)

return the logits ⟹ apply softmax to get the probabilities

compile your model architecture now.

L20 MNIST: Training Loop

- define the training config
  - → learning rate
  - → no. of epochs
  - → weight decay ⇒ L2 regularizaⁿ to avoid overfits
- define your loss funcⁿ & optimizer
- set the funcⁿ for training loop.
  - then for every mini-batch the model gets trained.
  - → after every batch, calculate the loss & accuracy. ⇒ keep updating the weights & zeroing out the gradients.

L21 Validaⁿ Loop : MNIST

- define the model.eval() ⇒ disables dropouts
  - ↳ disable the gradient computaⁿ so that we have time efficiency.
- keep all values ⇒ it can later create confusion matrix to get recall, paccision & F1.

L22 MNIST : Model Training

- now everything is defined, we can start our main training loop.
- iterate btw. training & evaluation loop to check how the model gets better
- define early stopping as per the requirements.
- later, see using visualization, how it looks.
- overfit = train_accuracy - test_accuracy. keep a threshold to check whether overfits happens or not.
- just remember, by flattening you have forgotten all the spatial info. This will get solved when we use CNNs.